

Guia tecnica: representacion fisica

Proyecto Fenomenos Atmosfericos - Three.js

Enfoque del documento: explicar unicamente la parte de atardecer.

Archivos principales: src/shaders/OndasShader.js, src/scenes/SunsetGlobalScene.js, src/scenes/SunsetLocalScene.js.

Objetivo: conectar el codigo con las formulas fisicas usadas para representar el cielo que se vuelve naranja/rojo cuando el Sol esta cerca del horizonte.

1. Alcance de la guia

Esta guia no documenta todos los fenomenos atmosfericos del proyecto. Se centra en la escena de atardecer, tanto en escala global como en escala local.

La implementacion combina un modelo fisico simplificado con decisiones de optimizacion. La parte fisica aparece principalmente en el shader espectral de Rayleigh; la parte de optimizacion aparece en los colores, opacidades, particulas, halos, trayectoria solar y la luz.

La idea central del atardecer es esta: cuando el Sol esta bajo, la luz atraviesa la atmosfera. En ese trayecto largo, las longitudes de onda cortas, sobre todo el verde, se dispersan mas. La luz que llega con mas fuerza al observador es la enriquecida en longitudes de onda largas: naranjas y rojos.

2. Archivos responsables

OndasShader.js define createRayleighSpectralMaterial(). Ese material se utiliza para la atmosfera y el cielo con una aproximacion de dispersion espectral.

SunsetGlobalScene.js muestra el fenomeno a escala planetaria: Tierra, atardecer, observador, rayos de color, punto de entrada a la atmosfera, arco de refraccion atmosferico y particulas de dispersion.

SunsetLocalScene.js muestra lo que ve una persona desde la superficie: el Sol bajo, halo, brillo del horizonte, niebla/fondo, luces ambientales, ciudad y particulas atmosfericas.

3. Formula fisica principal: ley de Rayleigh

La dispersion de Rayleigh establece que la intensidad dispersada por moléculas comparadas con la longitud de onda es aproximadamente proporcional a $1/\lambda^4$.

Formula conceptual: $I_R(\lambda)$ proporcional a $1/\lambda^4$.

Consecuencia: una longitud de onda corta se dispersa mucho mas que una larga. El azul se pierde del rayo directo con mas facilidad y el rojo permanece mas visible al final del recorrido.

En el shader, las longitudes de onda se codifican en micrometros aproximados para el verde y azul:

```
const vec3 lambda = vec3(0.680, 0.550, 0.440);
vec3 betaR = uRayleighStrength * (1.0 / pow(lambda, 4));
betaR *= 0.000015;
```

El vector lambda representa R=680 nm, G=550 nm y B=440 nm. Como el azul es de longitud de onda menor, $1/\lambda^4$ produce un coeficiente mayor para azul. En la

El azul se dispersa mas facilmente.

Esta es la formula mas importante para justificar fisicamente el color del proyecto.

4. Funcion de fase de Rayleigh

Ademas del factor espectral, el shader usa una funcion de fase. Esta describe reparte angularmente la luz dispersada segun el angulo entre la direccion de direccion del Sol.

```
float rayleighPhase(float mu) {  
    return 3.0 / (16.0 * PI) * (1.0 + mu * mu);  
}  
  
float mu = dot(viewDir, sunDir);  
float rPhase = rayleighPhase(mu);
```

mu es el coseno del angulo entre la vista de la camara y la direccion solar. Cuando el angulo cambia, cambia tambien la cantidad de luz dispersada que llega a la camara. Durante el atardecer, esta funcion ayuda a que el color atmosferico dependa de la longitud de onda, sino tambien de la geometria de observacion.

5. Aporte de Mie en el atardecer

El codigo tambien incluye dispersion de Mie. Rayleigh explica muy bien la dispersión de la luz azul/rojo por longitud de onda, mientras que Mie ayuda a representar el halo de brillo alrededor del Sol.

Mie se asocia a particulas mas grandes que las moleculas, como aerosoles. En la escena no se simula cada particula real; se usa como componente simplificada.

```
float miePhase(float mu, float g) {  
    float g2 = g * g;  
    return 3.0 / (8.0 * PI) * ((1.0 - g2) * (1.0 + mu * mu) +  
        (2.0 + g2) * pow(1.0 + g2 - 2.0 * g * mu, 1.5));  
}  
  
vec3 betaM = uMieStrength * vec3(1.0);  
float mPhase = miePhase(mu, uMieG);
```

En el atardecer local, uMieStrength aumenta cuando la elevacion solar baja, lo que refuerza el brillo calido cerca del horizonte.

6. Densidad atmosferica simplificada

El shader calcula una altura relativa dentro de la atmosfera y aplica una densidad exponencial de densidad. La densidad mayor cerca de la superficie aumenta con la altura visible.

```
float height = max(length(vWorldPos) - uPlanetRadius, 0.0);  
float atmH = max(uAtmosphereRadius - uPlanetRadius, 0.0);  
float h = clamp(height / atmH, 0.0, 1.0);  
float density = exp(-h * uDensityFalloff);
```

Interpretacion: cuando h es pequeno, el punto esta cerca de la superficie de la atmosfera. Cuando h se acerca a 1, el punto esta cerca del borde superior de la atmosfera, donde la densidad baja.

Para el atardecer esto importa porque la luz baja atraviesa capas densas de atmosfera.

py distancia aparente.

py7. Color final del shader

py El shader combina Rayleigh, Mie, densidad, intensidad solar y exposición para la dispersion total.

```
py vec3 scatter = uSunIntensity * density * (betaR * r + betaM * r^2);
py scatter *= uExposure;
py vec3 color = 1.0 - exp(-scatter);
```

py La expresion $1 - \exp(-\text{scatter})$ funciona como una compresion tonal: evita que crezca infinitamente y produce una respuesta mas natural para pantalla.

py Luego se agregan refuerzos visuales de azul y rojo. En atardecer se enfatiza el rojo/naranja mediante uRedBoost y factores de horizonte.

```
py color.b += (1.0 - sunset) * uBlueBoost * (1.0 - horizon);
py color.r += sunset * uRedBoost * horizon * 0.7 * sunset;
```

py8. Atardecer en escala global

py SunsetGlobalScene.js representa el fenomeno desde fuera del planeta. La distancia al sol es el radio EARTH_RADIUS = 3.0 y la atmosfera ATM_RADIUS = 3.42. Esa diferencia es el halo visible alrededor del planeta.

py La atmosfera se crea con createRayleighSpectralMaterial(). Es decir, el color del halo atmosferico depende del shader que contiene la ley $1 / \lambda^4$.

```
py this.EARTH_RADIUS = 3.0;
py this.ATM_RADIUS = 3.42;
py
py this.atmosphereMaterial = createRayleighSpectralMaterial({
py   sunDirection: this.sunDirection,
py   sunIntensity: 11.0,
py   alpha: 0.72,
py   exposure: 1.25,
py   blueBoost: 1.3,
py   redBoost: 1.4,
py   side: THREE.BackSide,
py });
```

py En esta escala se busca que el estudiante vea la causa geometrica: Sol bastante lejano y un recorrido atmosferico largo antes de llegar al observador.

py9. Hora solar y elevacion

py La hora controla el angulo solar. Para el tramo diurno 6h a 18h, la elevacion se calcula con una senoide: 0 al amanecer, 1 al mediodia y 0 al atardecer.

```
py const t = Math.max(0, Math.min(1, (hour - 6) / 12));
py const elevation = Math.sin(t * Math.PI);
```

py En atardecer, cerca de las 18h, elevation se aproxima a 0. El codigo usa uRedBoost para reforzar los efectos propios del atardecer: mas rojo, mas particulas y mayor recorrido atmosferico.

py10. Posicion del Sol en la escena global

py El Sol se mueve en el plano XZ. A las 12h esta alto respecto al observador y a las 6h y 18h queda cerca del horizonte visual.

```
py const sunAngle = ((hour - 6) / 12) * Math.PI;
```

```

    py const x = Math.sin(sunAngle) * 14.0;
    py const z = -Math.cos(sunAngle) * 12.0;
    py return new THREE.Vector3(x, 0, z);

```

Esta geometria no pretende reproducir la astronomia exacta de la Tierra, relacion clara entre hora, posicion del Sol y recorrido de la luz.

11. Entrada del rayo a la atmosfera

Para mostrar donde la luz solar toca la atmosfera antes de llegar al observador, el codigo calcula la interseccion entre un rayo y una esfera.

La esfera es la atmosfera y el rayo va desde la posicion del Sol hacia la del observador.

```

    py const rayDir = obsPos.clone().sub(sunPos).normalize();
    py const b = 2 * sunPos.dot(rayDir);
    py const c = sunPos.dot(sunPos) - atmRadius * atmRadius;
    py const disc = b * b - 4 * c;
    py const t1 = (-Math.sqrt(disc) - b) / 2;
    py return sunPos.clone().add(rayDir.clone().multiplyScalar(t1));

```

Fisicamente, esto representa el punto donde la luz entra en la capa atmosférica.

Pedagogicamente, permite dibujar el segmento Sol-atmosfera y luego atmosfera-observador.

12. Recorrido atmosferico largo

La clave del atardecer es que el camino dentro de la atmosfera aumenta cuando la elevacion solar baja. El codigo lo representa con una aproximacion proporcional a la elevacion.

```

    py const angleBetween = obsDir.angleTo(sunDir);
    py const elevAngle = Math.PI / 2 - angleBetween;
    py const elevNorm = Math.max(0, Math.sin(elevAngle));
    py
    py const pathLength = elevNorm > 0.01
    py ? atmR / Math.max(elevNorm, 0.05)
    py : atmR * 18;

```

Cuando elevNorm es pequeno, el divisor es pequeno y pathLength crece. Esto representa la geometria del color rojizo: el rayo pasa por mas atmosfera y pierde mas por dispersion.

La escena dibuja un arco y una barra para que este aumento no quede solo en el codigo, sino como elemento visual.

13. Rayos de longitud de onda

La escena global usa tres rayos: azul, verde y rojo. No son fotones reales, sino representaciones didacticas de tres rangos de longitud de onda.

En atardecer, el azul se corta pronto, el verde llega parcialmente y el rojo llega al observador.

```

    py const blueReach = entryPoint.clone().lerp(obsPos, 0.1);
    py this._setLinePoints(this.blueIntrRay, entryPoint, blueReach);
    py
    py const greenReach = entryPoint.clone().lerp(obsPos, 0.5);
    py this._setLinePoints(this.greenIntrRay, entryPoint, greenReach);
    py
    py this._setLinePoints(this.redIntrRay, entryPoint, obsPos);

```

Esto traduce la ley de Rayleigh a una imagen: como azul se dispersa mas que el verde y el rojo.

llegando completo al observador. Rojo, al dispersarse menos, se mantiene directo.

14. Opacidad e intensidad durante el atardecer

El código aumenta el peso visual del rojo cuando elevation baja. Esta parte de la decisión visual basada en la física, no una ecuación atmosférica exacta.

```
const redStrength = 0.55 + (1 - elevation) * 0.45;
this.redIntRay.material.opacity = redStrength * nightFade;
this.blueIntRay.material.opacity = (0.08 + elevation) * nightFade;
this.greenIntRay.material.opacity = (0.15 + elevation) * nightFade;
```

Cuando elevation es baja, 1 - elevation es alto. Por eso el rojo gana opacidad al atardecer.

nightFade evita que los rayos sigan brillando después de la ventana de atardecer. En el código global, después de las 20h la intensidad cae casi por completo.

15. Partículas de dispersión

Las partículas son una capa pedagógica. Ayudan a mostrar que la luz azul se dispersa en la atmósfera, mientras que el rojo puede conservar una dirección hacia el observador durante la ventana de atardecer.

```
this.blueScatter = makeScatter(0x4488ff, 180);
this.greenScatter = makeScatter(0x33cc66, 90);
this.redScatter = makeScatter(0xff3311, 140);
```

El ancho de la nube aumenta cuando baja la elevación solar:

```
const spreadBlue = 0.8 + (1 - elevation) * 1.2;
const spreadGreen = 0.4 + (1 - elevation) * 0.6;
const spreadRed = 0.3 + (1 - elevation) * 0.4;
```

16. Ventana temporal del atardecer

El comportamiento rojo especial se concentra alrededor del atardecer. En SunsetGlobalScene.js se calcula un factor que crece de 16h a 18h y decrece después.

```
let pFactor = 0.0;
if (hour >= 16 && hour <= 20) {
    pFactor = hour <= 18 ? (hour - 16) / 2 : (20 - hour) / 2;
}
this._sunsetParticleFactor = pFactor;
```

A las 18h el factor vale 1. Eso representa el punto máximo del atardecer. A las 20h vuelve a 0 para indicar que ya no se está representando el atardecer.

17. Escena local del atardecer

SunsetLocalScene.js representa la experiencia humana desde la superficie. Se usa createRayleighSpectralMaterial(), pero aplicado a un domo de cielo grande para que la cámara quede dentro.

```
const domeGeo = new THREE.SphereGeometry(220, 64, 32);
this.skyMaterial = createRayleighSpectralMaterial({
    sunIntensity: 12.0,
    rayleighStrength: 1.3,
    mieStrength: 0.003,
    densityFalloff: 1.15,
    planetRadius: 180.0,
    atmosphereRadius: 220.0,
```

```

    py redBoost: 1.5,
    py side: THREE.BackSide,
    py } );

```

La escala local se apoya mas en sensaciones visuales: color de cielo, halo, de horizonte, luces calidas, niebla y oscurecimiento progresivo.

18. Refuerzo local del color naranja/rojo

En la escena local, la elevacion solar controla el color del Sol, el halo, el suelo, la niebla y el fondo.

```

    py this.skyMaterial.uniforms.uBlueBoost.value = (0.6 -
    py progress);
    py this.skyMaterial.uniforms.uRedBoost.value = (2.2 -
    py press);
    py this.skyMaterial.uniforms.uSunIntensity.value = (8.
    py Progress * 0.98);
    py this.skyMaterial.uniforms.uMieStrength.value = (0.0
    py 0 - nightProgress);

```

Durante atardecer, elevation baja. Entonces uRedBoost aumenta y uMieStrength aumenta. Eso produce un horizonte mas calido y con mas halo/bruma.

19. Brillo del horizonte

El horizonte es donde mas se refuerza el color calido porque visualmente es la zona donde la luz solar llega rasante.

```

    py const sunsetGlow = Math.max(0, 1.0 - elevation * 2.
    py this.horizonGlowMat.opacity = sunsetGlow * 0.55;

```

Si elevation es baja, sunsetGlow aumenta. Cuando llega la noche, nightProgress ese brillo para que el naranja no permanezca indefinidamente.

20. Diferencia entre fisica y recurso visual

Parte fisica: betaR proporcional a $1 / \lambda^4$, funcion de fase de Rayleigh, fase de Mie, densidad exponencial y geometria del recorrido atmosferico.

Parte visual: blueBoost, redBoost, colores discretos del Sol, opacidades de partículas, halos, brillo del horizonte, color del suelo y ventanas de tiempo.

La representacion es correcta para explicar el principio: en atardecer hay dispersión atmosferica, se dispersa mas el azul y domina el rojo/naranja. No es una dispersión atmosferica completa con integracion volumetrica real, absorcion de ozono, rebotes o aerosoles medidos.

21. Resumen para exposicion

La parte del atardecer se basa en la dispersion de Rayleigh: las longitudes cortas se dispersan mas porque la intensidad dispersada depende de $1 / \lambda^4$.

En el codigo, esa relacion aparece en OndasShader.js con $\beta_R = 1 / \lambda^4$. El azul tiene menor longitud de onda, por eso su coeficiente de dispersion es mayor.

SunsetGlobalScene.js muestra la causa global: cuando el Sol esta cerca del horizonte, el rayo atraviesa mas atmosfera antes de llegar al observador.

SunsetLocalScene.js muestra el efecto observado: cielo naranja/rojo, halo calido del horizonte y oscurecimiento progresivo.

Los elementos visuales refuerzan la explicacion fisica: azul se dispersa mas.

El rojo llega parcialmente y el rojo llega con mayor presencia al observador.